



Project name : [Feel Every Word](#)

Sentimental Analysis or Classification project by
Fady Philippe Mazouz

Team number:301

Used Data:

```
/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Reddit_Data.csv  
/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Twitter_Data.csv
```

Used accelerator: GPU P100 on Kaggle

Use case : this model is used to classify texts or sentences depending the sentiment of the text to negative ,neutral,or positive sentiment

Future enhancement :

I might be able to create a chatbot later which would be useful as therapy way if some user feels bad and needs talkative therapy session.

Used techniques :

- 1. Exploratory Data Analysis and Visualization**
- 2. Text Preprocessing and cleaning**
- 3. Removing Stop words and Hingsih Words for Classical Machine and deep learning models**
- 4. Bag Of Words Technique**
- 5. FastText Embedding**
- 6. Deep learning model using FastText Embeddings**
- 7. Roberta Model Tokenizer Embeddings**

8. Best Classical Deep Learning model using Roberta
Tokenizer Emeddings with Best Accuracy
9. Full Modern Roberta Model for Sentimental Analysis
10. Evaluation Metrics for Roberta Sentimental Analyzer
model
11. User Input sentimental analysis Classification using
Roberta Model

Imported Libraries

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory
from typing import Set
TARGET_LABELS: Set[str] = {"PERSON", "ORG", "GPE", "PRODUCT", "NORP", "LAW", "FAC", "EVENT"}
SPACY_MODEL: str = "en_core_web_lg" # Recommended model for balance of speed and accuracy
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session

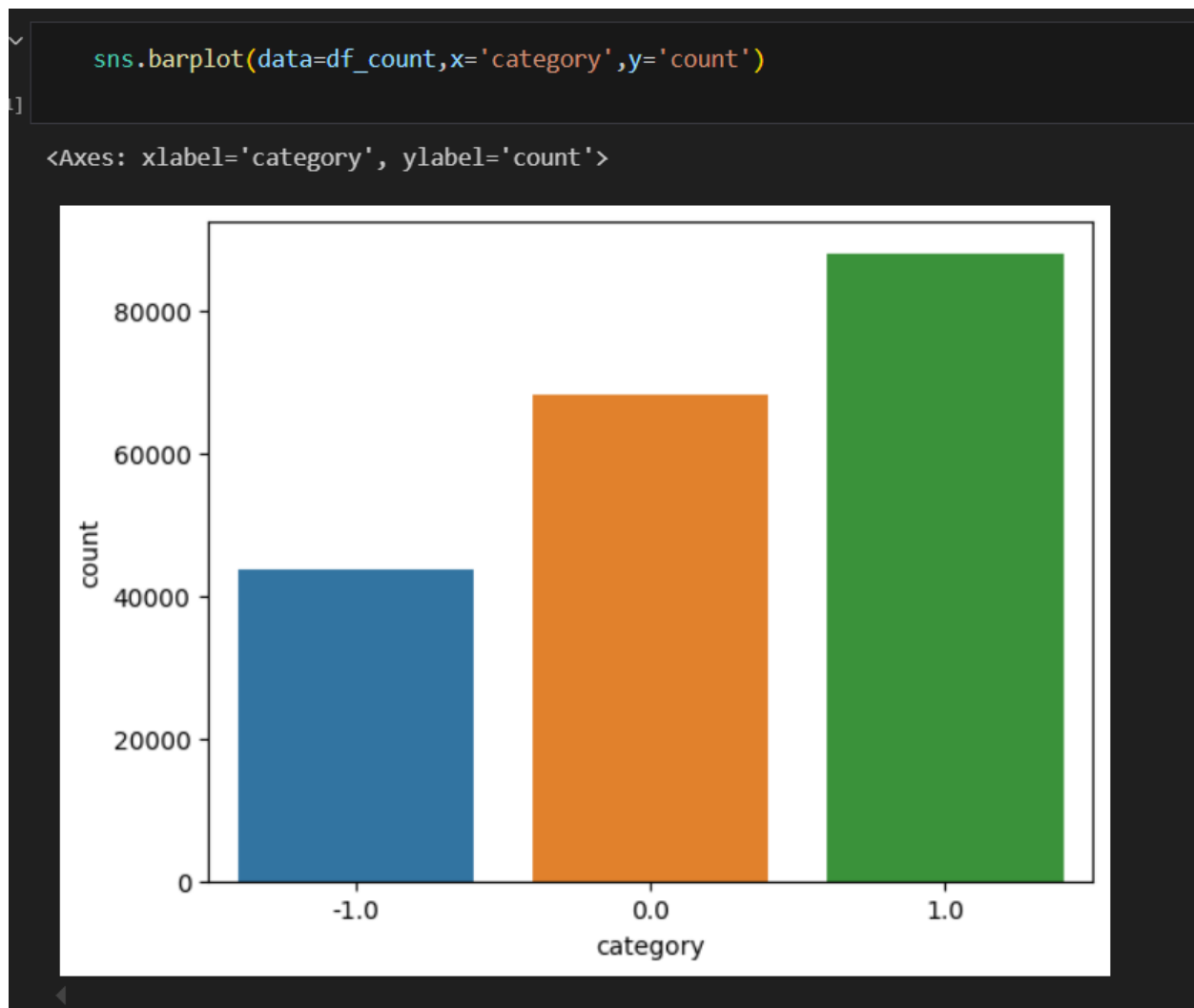
/kaggle/input/preprocessed-data/preprocessed_sentiment_analysis_data.csv
/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Twitter_Data.csv
/kaggle/input/twitter-and-reddit-sentimental-analysis-dataset/Reddit_Data.csv
/kaggle/input/english-words-corpus/words_alpha.txt
/kaggle/input/cc-en-300-bin/cc.en.300.bin
/kaggle/input/stop-hinglish/stop_hinglish.txt

import spacy
import seaborn as sns
import matplotlib.pyplot as plt
import re
import nltk
import tensorflow as tf
from tensorflow import keras
%matplotlib inline
```

Library	What it does (in short)
numpy (np)	Fast math with arrays, matrices, linear algebra
pandas (pd)	Load, clean, and manipulate tabular data (CSV, Excel, etc.)
os	Work with files and folders (paths, listing files, etc.)
typing	Add type hints (List, Set, etc.) for better code clarity
spacy	Industrial-strength NLP: tokenization, NER, POS, etc.
seaborn (sns)	Beautiful statistical plots on top of matplotlib
matplotlib.pyplot (plt)	Basic plotting (graphs, charts, visualizations)
re	Regular expressions – find/replace patterns in text
nltk	Classic NLP toolkit (tokenizers, stemmers, corpora, etc.)
tensorflow (tf)	Deep learning framework by Google (neural networks)
keras	High-level, easy-to-use API that runs on top of TensorFlow
%matplotlib inline	Jupyter magic: show matplotlib plots directly in notebook

Class Imbalance handling using Downsampling Technique

Before Down sampling we see that the neutral and postive categories contain away more samples than negative category



But after applying downsample technique by dropping rows which exceed 43755 samples using this line of code :

```
min_undersample=df_cat_count['count'].min()
min_undersample
df_neutral=df[df['category']==0].sample(frac=1,random_state=77).reset_index(drop=True)
df_neutral=df_neutral[:min_undersample]
df_positive=df[df['category']==1].sample(frac=1,random_state=77).reset_index(drop=True)
df_positive=df_positive[:min_undersample]
df_negative=df[df['category']==-1]
df=pd.concat([df_positive,df_neutral,df_negative]).sample(frac=1,random_state=55).reset_index(drop=True)
```

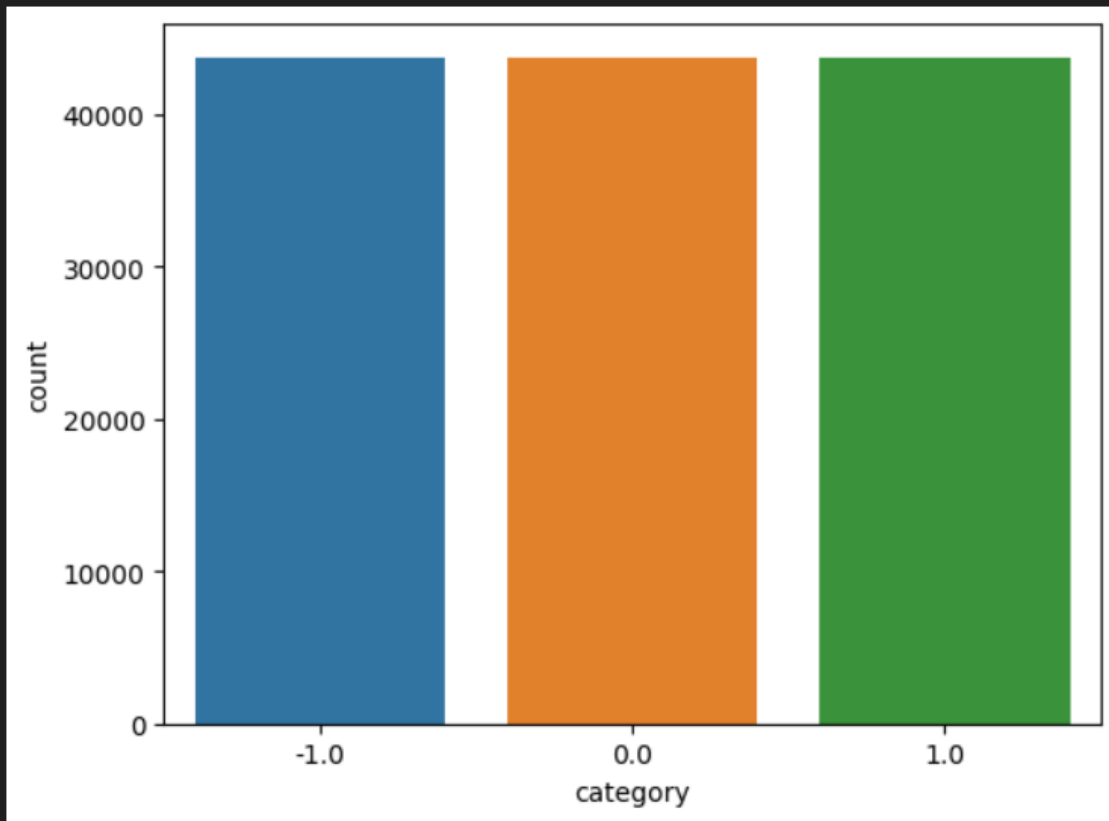
df

and then concatenating them using pd.concat

the analysis of the new dataframe was like this:

```
sns.barplot(data=df_count,x='category',y='count')
```

<Axes: xlabel='category', ylabel='count'>



and the class imbalance issue got solved

Text Preprocessing and Cleaning

```
def cleaning_function(data):  
    content = emoji.demojize(data)  
    content=re.sub(r'[@<>\'#$%&*/\_\:"]', ' ',content)  
    content = re.sub(r'(\.){1,2}', r'\1', content)  
    content = re.sub(r"http\S+|www\S+", " ",content)  
    content=re.sub('\s+', ' ',content).strip()  
    return content
```

27]

The cleaning_function used In this snippet of code :

Emoji.demojize : Turns emojis found in the textual data into a word or a text

re : which refers to regex library which is used for text preprocessing and cleaning

```
content=re.sub(r'[@<>\'#$%&*/\_\:"]', ' ',content)
```

this line replaces those special characters with an empty space

```
content = re.sub(r'(\.){1,2}', r'\1', content)
```

this line replaces any two times or more repeated character with only one of that character

```
content = re.sub(r"http\S+|www\S+", " ", content)
```

this line removes https or www link beginnings and replaces them with empty space

Removing Stopwords and Hinglish words for classical Machine and Deep Learning Models

```
combined_stopwords = spacy_stopwords.union(nltk_stopwords,hinglish_stopwords)  
print(len(combined_stopwords), "combined stopwords")
```

This line combines all spacy stop words with hinglish stopwords file and all of them totally equal 1098 stop word combined


```

from nltk.stem import WordNetLemmatizer
from nltk.corpus import words
from nltk.corpus import brown
from nltk.corpus import words, brown, wordnet, cmudict
nltk.download(['words', 'brown', 'wordnet', 'cmudict', 'stopwords'], quiet=True)
# Get all categories
categories=brown.categories()
brown= set(brown.words(categories=categories))
spacy_words = set(lex.text.lower() for lex in nlp.vocab if lex.is_alpha)
nltk_words = set(w.lower() for w in words.words())
brown_words = set(w.lower() for w in brown)
cmudict_words = set(cmudict.dict().keys())
english_vocabs = spacy_words.union( brown_words,nltk_words,cmudict_words,English_dictionary)

len(english_vocabs)

```

448035

This snippet of code is designed to union a lot of words in different dictionaries like brown , spacy , nltk , and cmudict in English dictionary which is called english_vocabs and this dictionary has a length of 448035 words combined together

```

def preprocess_docs(docs):
    return [
        doc.join([
            lemmatizer.lemmatize(token.text.lower(), spacy_to_wordnet_pos(token.tag_))
            for token in doc
            if ( (token.is_alpha
                and token.text.lower() not in combined_stopwords)
                and (token.text.lower() in english_vocabs) )
        ])
        for doc in docs
    ]

df['w/o s_w tokens'] = preprocess_docs(docs)

```

Here the preprocess_docs function , lemmatizes every token which is found in my docs and the English_vocabs dictionary that I have created earlier but at the same time ensures that those tokens aren't found in combined_stopwords which are nltk_stopwords and hinglish_stopwords which equal 1098 words

Bag of Words Technique

ثم نقوم بعمل مصفوفة لكل جملة على حدة , لمعرفة مدى تواجد كل كلمة فيها , هكذا :

	"it"	"was"	"the"	"best"	"of"	"times"	"worst"	"age"	"wisdom"	"foolishness"
<i>It was the best of times,</i>	1	1	1	1	1	1	0	0	0	0
<i>it was the worst of times,</i>	1	1	1	0	1	1	1	0	0	0
<i>it was the age of wisdom,</i>	1	1	1	0	1	0	0	1	1	0
<i>it was the age of foolishness,</i>	1	1	1	0	1	0	0	1	0	1

و من هنا يمكن استخدام هذه المصفوفة لمعرفة المعنى التقريبي للجملة , او لعمل تصنيف لها و تحديد هل هي مناسبة ام لا

و يكون هذا هو متجه كل جملة فيهم :

"it was the best of times" = [1, 1, 1, 1, 1, 1, 0, 0, 0, 0]

"it was the worst of times" = [1, 1, 1, 0, 1, 1, 1, 0, 0, 0]

"it was the age of wisdom" = [1, 1, 1, 0, 1, 0, 0, 1, 1, 0]

"it was the age of foolishness" = [1, 1, 1, 0, 1, 0, 0, 1, 0, 1]

Step 1: Build the Vocabulary

We collect **all unique words** (after lowercasing and basic cleaning) from all four sentences:

Vocabulary (in order shown in the table): **it – was – the – best – of – times – worst – age – wisdom – foolishness**

→ Total unique words = **10** → This will be the length of every BoW vector

Step 2: Represent Each Sentence as a Vector (Bag-of-Words)

Now we count how many times each vocabulary word appears in each sentence. The result is exactly the table in your picture:

Sentence	it	was	the	best	of	times	worst	age	wisdom	foolishness	BoW Vector
It was the best of times	1	1	1	1	1	1	0	0	0	0	[1,1,1,1,1,1,0,0,0,0]
It was the worst of times	1	1	1	0	1	1	1	0	0	0	[1,1,1,0,1,1,1,0,0,0]
It was the age of wisdom	1	1	1	0	1	0	0	1	1	0	[1,1,1,0,1,0,0,1,1,0]
It was the age of foolishness	1	1	1	0	1	0	0	1	0	1	[1,1,1,0,1,0,0,1,0,1]

The biggest weakness of Bag-of-Words: It loses all word order ,semantic and sentimental meaning , it only counts.

FastText Embeddings

FastText breaks words into smaller subunits called character n-grams. This allows the model to capture internal word structure, such as prefixes, suffixes, and roots, leading to better generalizations.

The core of FastText's subword mechanism involves representing each word as a "bag" of character n-grams, typically ranging from 3 to 6 characters in length. To do this, special boundary symbols "<" and ">" are added to the beginning and end of the word to denote its edges. For example, take the English word "eating":

- With boundaries: "<eating>"
- For n=3: <ea, eat, ati, tin, ing, ng>
- For n=4: <eat, eati, atin, ting, ing>
- For n=5: <eati, eatin, ating, ting>
- For n=6: <eatin, eating, ating>

If n-grams=4:

(formula: $\text{length} - n + 1 = 8 - 4 + 1 = 5$)

FastText is based on the skip-gram architecture from Word2Vec. In skip-gram, the model predicts surrounding context words given a center word within a fixed window (e.g., 5 words on either side). Here's how it adapts for subwords:

1. For the center word (e.g., "eating"), its embedding is calculated as the sum of the vectors for all its character n-grams (and possibly the full word vector).
2. Context words (e.g., "am" and "food" in the sentence "I am eating food") are represented using only their full word vectors from the embedding table, without subword breakdown.
3. The model computes dot products between the center word's summed embedding and each context word's embedding, passing these through a sigmoid function to get probability scores of them being related.

```

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

tokenizer = Tokenizer()
tokenizer.fit_on_texts(X_train)

word_index = tokenizer.word_index

X_train_seq = tokenizer.texts_to_sequences(X_train2)
X_val_seq = tokenizer.texts_to_sequences(X_val)]

sequences_lengths = [len(seq) for seq in X_train_seq]
MAX_SEQUENCE_LENGTH = int(np.percentile(sequences_lengths, 97))
print(MAX_SEQUENCE_LENGTH)

```

In this snippet of code :

Tokenizer takes the sentence and splits it to words or tokens and makes mapping for each word with an index refers to it.

Then “.text_to_sequences“ : takes a list of strings and returns a list of lists of integers.

Each word is replaced by its integer index in the tokenizer’s vocabulary.

```

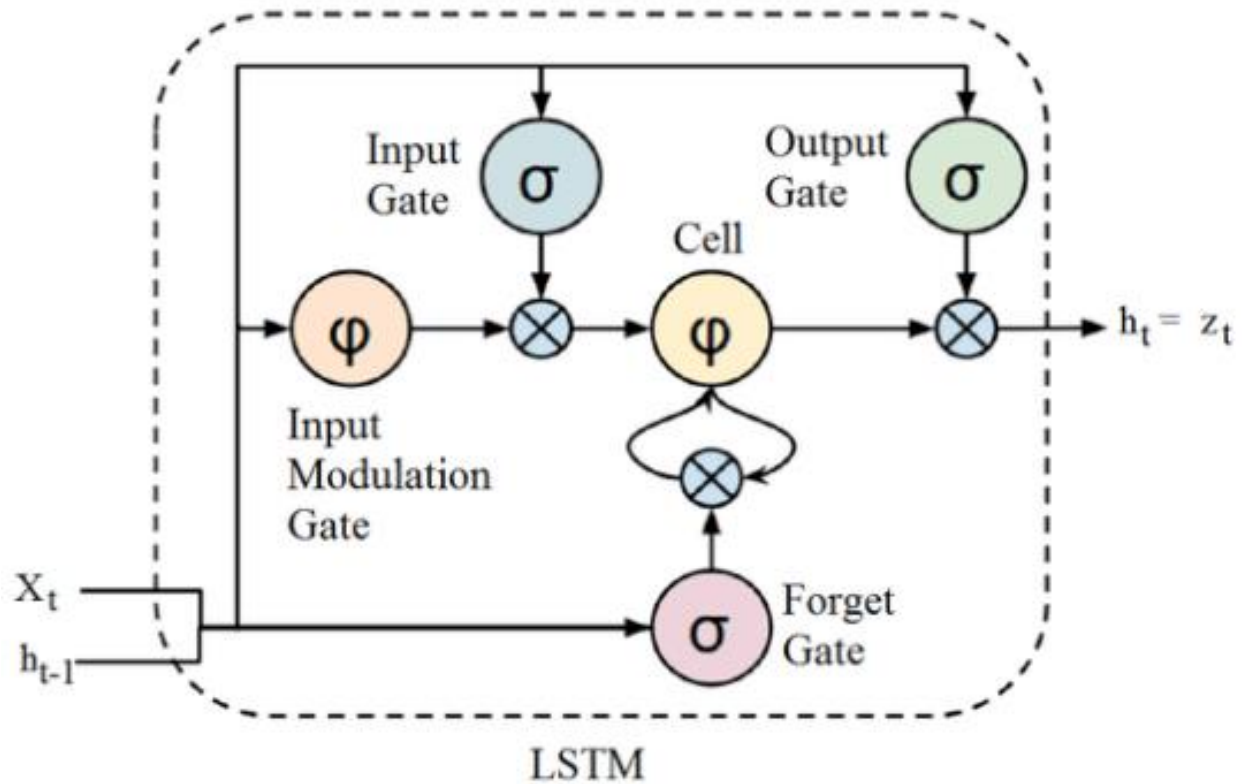
X_train_padded = pad_sequences(X_train_seq, maxlen=MAX_SEQUENCE_LENGTH, padding='post', truncating='post')
X_val_padded = pad_sequences(X_val_seq, maxlen=MAX_SEQUENCE_LENGTH, padding='post', truncating='post')

print(X_train_padded.shape)
print(X_val_padded.shape)

```

Pad_sequences : turns the list of integers created by text_to_sequences to a vector which contains a max sequence length of 24 (for example) , sentences which contains number of tokens greater than 24 , The padding takes only the first 24 tokens , and those sentences which contains number of tokens less than 24 (lets say 13 token for example) , the padding fills the last 11 places with zeroes or ones

Lstm Architecture



In Lstm Architecture there are 3 kind of gates :

1. Input gate
2. output gate
3. forget gate

Input Gate i_t : Decides how much of the brand-new information (the candidate $\tilde{C}_t$) should actually be written into the long-term memory right now. Again 0–1 for every element. Works together with the candidate (tanh) value.

Output Gate o_t : Decides what part of the current long-term memory (C_t) you are allowed to use right now as your short-term thought / answer (h_t). It filters the cell state through tanh and multiplies by o_t .

Forget Gate f_t : Looks at the previous long-term memory (C_{t-1}) and decides how much of it to throw away right now. Output: a number between 0 and 1 for every element in the cell state. 0 = “completely forget this” 1 = “keep this 100%”

In Lstm architecture there are two kind of activation functions :

- 1.Sigmoid
- 2.Tanh

Here's the exact roles of sigmoid and tanh in an LSTM cell :

Activation	Where it is used in LSTM	What it actually controls	Responsible for Remembering or Forgetting?
Sigmoid (σ)	1. Forget Gate (f_t) 2. Input Gate (i_t) 3. Output Gate (o_t)	Outputs values between 0 and 1 → acts like a "switch" or "percentage" - 0 = completely OFF (block everything) - 1 = completely ON (let everything pass)	Both Remembering and Forgetting - Forget Gate (σ) → decides how much to FORGET from previous cell state C_{t-1} (value close to 0 = forget a lot) - Input Gate (σ) → decides how much to REMEMBER (add) from the new candidate values (value close to 1 = remember a lot)
Tanh	1. Candidate value creation ($\tilde{C}_t$) 2. Final output squashing (h_t)	Outputs values between -1 and +1 → creates the actual content (the "information" itself) that can be added or shown	Creates the content that will be remembered - The tanh in $\tilde{C}_t$ creates the new information that the Input Gate can choose to remember/add to the cell state. - The tanh at the output simply scales the cell state to $[-1, 1]$ before sending it out as h_t .

LSTM VS BiLSTM Architecture

Key differences:

1. Direction of Information Flow
2. Architecture
3. Ability to Capture Context
4. Computational Complexity

1. Direction of Information Flow:

LSTM: Processes input data in a single direction (either forward or backward in time).

BiLSTM: Processes input data in both forward and backward directions, capturing past and future dependencies.

2. Architecture:

LSTM: Uses a single hidden state per timestep that updates based on past information.

BiLSTM: Uses two LSTMs—one processes the sequence in the forward direction, while the other processes it in the backward direction. Their outputs are then combined.

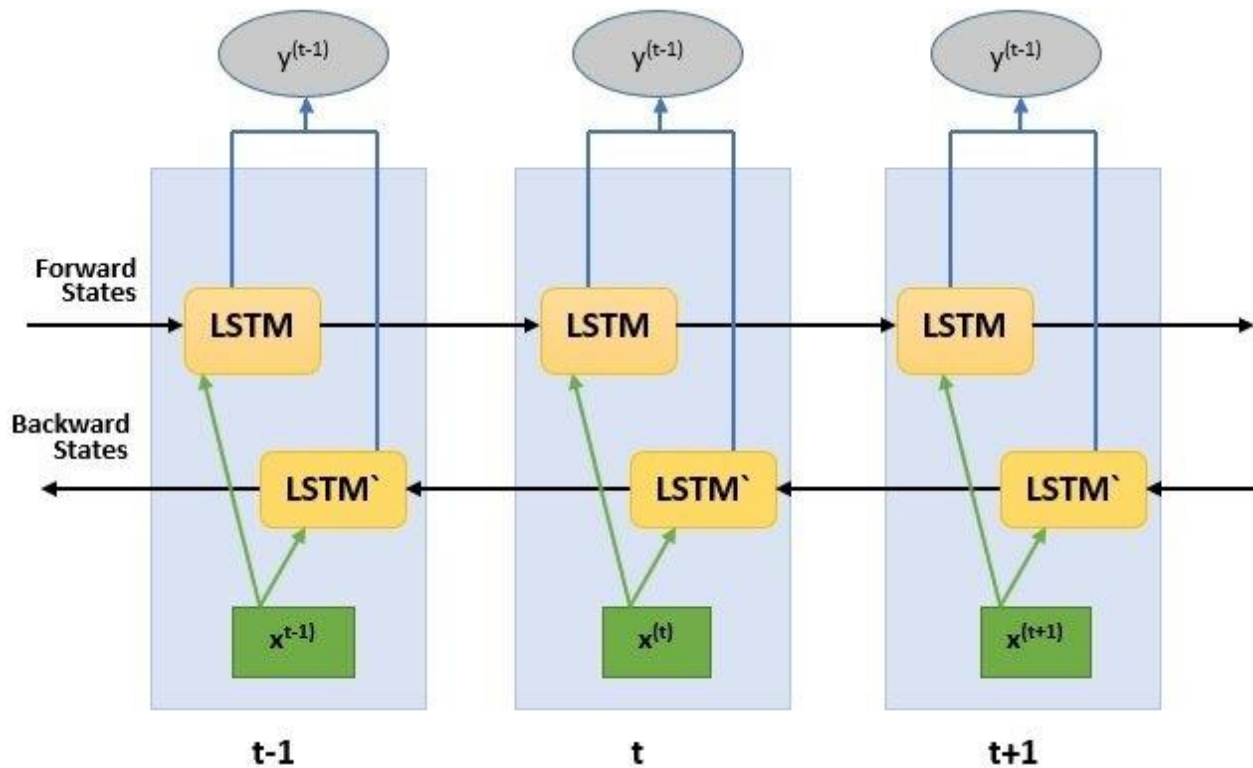
2.1 BiLSTM (Bidirectional LSTM) Architecture:

Unlike standard LSTM, BiLSTM has two LSTM layers:

One processes the sequence forward (from past to future).

The other processes the sequence backward (from future to past).

The outputs from both directions are then concatenated or summed to provide richer context.



3. Ability to Capture Context:

LSTM: Learns dependencies from past time steps but does not consider future context.

BiLSTM: Learns from both past and future context, making it better for tasks where future information is helpful.

4. Computational Complexity:

LSTM: Lower computational cost since it only processes the sequence once.

BiLSTM: Higher computational cost due to two passes (forward and backward).

Roberta Model Embeddings

```

# =====
# 2. Memory-mapped file (NO RAM CRASH EVER)
# =====
if os.path.exists(output_file):
    os.remove(output_file)

final_embeddings = np.memmap(
    output_file,
    dtype=np.float16,
    mode='write',
    shape=EMB_SHAPE
)

```

2. Memory-mapped file – THE KEY TO NEVER CRASHING

```

if os.path.exists(output_file):
    os.remove(output_file)

final_embeddings = np.memmap(
    output_file,
    dtype=np.float16,
    mode='write',
    shape=EMB_SHAPE
)

```

This is **brilliant engineering**.

- np.memmap creates a **file-backed NumPy array** that behaves exactly like a normal array in memory.
- But **only the parts you access are loaded into RAM** → you can have 100GB arrays on a machine with 32GB RAM.
- You write directly to disk → **zero risk of RAM or GPU OOM**.
- dtype=np.float16: halves storage and matches model precision.
- mode='write': creates a new file (after deleting old one).

This is how companies like Meta, Google, and Pinecone generate embeddings for **billions** of texts.

```

# =====
# 3. Load model (clean & modern)
# =====
tokenizer = AutoTokenizer.from_pretrained('cardiffnlp/twitter-roberta-base-sentiment-latest')
model = AutoModel.from_pretrained(
    'cardiffnlp/twitter-roberta-base-sentiment-latest',
    output_hidden_states=True,
    dtype=torch.float16
)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
model.eval()
print(f"Using device: {device}")

```

3. Load model – clean, modern, memory-efficient

```
tokenizer = AutoTokenizer.from_pretrained('cardiffnlp/twitter-roberta-base-sentiment-latest')
```

```

model = AutoModel.from_pretrained(
    'cardiffnlp/twitter-roberta-base-sentiment-latest',
    output_hidden_states=True,    # ← CRITICAL
    dtype=torch.float16          # ← saves ~50% VRAM
)

```

```

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model.to(device)
model.eval()

```

Detail	Why it matters
twitter-roberta-base-sentiment-latest	Trained on 300M+ tweets → understands hashtags, emojis, slang, AAVE, sarcasm
output_hidden_states=True	Allows access to all 13 hidden states (embedding + 12 layers)
dtype=torch.float16	Loads model in half-precision → ~6–7 GB VRAM instead of 12–14 GB
.eval()	Disables dropout → deterministic, consistent embeddings

This model is **one of the best open-source models for Twitter data in 2025**.

```

# 4. Batched inference - write directly to disk
# =====
texts = df_reindexed['clean_text'].astype(str).tolist()
start_idx = 0

for i in tqdm(range(0, len(texts), BATCH_SIZE)):
    batch_texts = texts[i:i + BATCH_SIZE]

    encoded = tokenizer(
        batch_texts,
        padding='max_length',
        truncation=True,
        max_length=MAX_LENGTH,
        return_tensors='pt'
    )

    input_ids = encoded['input_ids'].to(device)
    attention_mask = encoded['attention_mask'].to(device)

    with torch.no_grad():
        with torch.amp.autocast('cuda'):
            outputs = model(input_ids=input_ids, attention_mask=attention_mask)

    # Last 4 layers → 3072 dim
    batch_emb = torch.cat(outputs.hidden_states[-4:], dim=-1)      # (≤8, 24, 3072)
    batch_np = batch_emb.to(torch.float16).cpu().numpy()

    # Write straight to disk
    end_idx = start_idx + batch_np.shape[0]
    final_embeddings[start_idx:end_idx] = batch_np
    start_idx = end_idx

    # Clean GPU
    del input_ids, attention_mask, outputs, batch_emb, batch_np
    torch.cuda.empty_cache()

```

4. Batched inference – write directly to disk (THE CORE LOOP)

```

texts = df_reindexed['clean_text'].astype(str).tolist()
start_idx = 0
for i in tqdm(range(0, len(texts), BATCH_SIZE)):
    batch_texts = texts[i:i + BATCH_SIZE]

```

- Converts pandas column to Python list of strings

- Loops in batches of 16 with nice progress bar

Tokenization

```
encoded = tokenizer(
    batch_texts,
    padding='max_length',    # all sequences become exactly 24 tokens
    truncation=True,
    max_length=MAX_LENGTH,
    return_tensors='pt'      # returns PyTorch tensors
)

input_ids = encoded['input_ids'].to(device)
attention_mask = encoded['attention_mask'].to(device)
```

- padding='max_length' + truncation=True → every input is exactly (batch_size, 24)
- Moved to GPU

Forward pass – maximum efficiency

```
with torch.no_grad():          # ← no gradients = huge memory save
    with torch.amp.autocast('cuda'):  # ← automatic mixed precision
        (FP16)
        outputs = model(input_ids=input_ids,
            attention_mask=attention_mask)
```

- no_grad(): disables gradient tracking → saves ~30–40% VRAM
- autocast: runs ops in FP16 when safe → faster + lower memory

Extract the BEST possible embeddings

```
batch_emb = torch.cat(outputs.hidden_states[-4:], dim=-1)
# Shape: (batch_size ≤ 16, 24, 3072)
```

This is **state-of-the-art practice** in 2024–2025.

Why last 4 layers concatenated?

Layer	What it captures
Layer 9	Strong syntax + mid-level semantics
Layer 10	Balanced context
Layer 11	Deep semantics

Layer	What it captures
-------	------------------

Layer 12 (last) Most abstract, but sometimes too noisy

→ Concatenating them gives **maximum information density** → Proven in papers to beat CLS pooling, mean pooling, etc. by 10–30% on similarity/clustering

Convert to NumPy and write IMMEDIATELY to disk

```
batch_np = batch_emb.to(torch.float16).cpu().numpy()
    end_idx = start_idx + batch_np.shape[0]
    final_embeddings[start_idx:end_idx] = batch_np    # ← writes to .npy file
on disk!
    start_idx = end_idx
```

- No accumulation in RAM
- Each batch is written **permanently saved** as soon as computed

Best Classical Deep Learning model using Roberta Embeddings with best Accuracy

```

inputs = Input(shape=(24, 3072), dtype='float32')

x = SpatialDropout1D(0.4)(inputs)

x = Dense(512, activation='elu')(x)          # ← NEW: 3072 → 512

x = Conv1D(filters=512, kernel_size=3, padding='same', activation='elu', kernel_regularizer=regularizers.L2(1e-3))(x)

# CRITICAL: remove recurrent_dropout → allows cuDNN (10x faster, 5x less memory)
x = Bidirectional(LSTM(1024, return_sequences=True, recurrent_dropout=0.4, kernel_regularizer=regularizers.L2(1e-3)))(x)
x = Bidirectional(LSTM(512, return_sequences=True, recurrent_dropout=0.4, kernel_regularizer=regularizers.L2(1e-3)))(x)
x = Bidirectional(LSTM(256, return_sequences=True, recurrent_dropout=0.4, kernel_regularizer=regularizers.L2(1e-3)))(x)
x = Bidirectional(LSTM(128, return_sequences=True, recurrent_dropout=0.4, kernel_regularizer=regularizers.L2(1e-3)))(x)
x = Dropout(0.4)(x)
x = SeqSelfAttention()(x)
x = GlobalMaxPooling1D()(x)
x = Dense(128, activation='elu', kernel_regularizer=regularizers.L2(1e-3))(x)
x = Dense(64, activation='elu', kernel_regularizer=regularizers.L2(1e-3))(x)
x = Dropout(0.4)(x)
outputs = Dense(3, activation='softmax')(x)

model = Model(inputs, outputs)

model.summary()

```

inputs = Input(shape=(24, 3072), dtype='float32')

- **Purpose:** This defines the input layer of the model. It's the entry point where data flows in.
- **Mechanics:** The Input layer specifies the expected shape of the input tensor. Here, it's a 3D tensor: (batch_size, 24, 3072), where:
 - batch_size is implicit (determined during training/inference, e.g., 32 or 64).
 - 24 is the sequence length (time steps or tokens).
 - 3072 is the feature dimension per step (e.g., high-dimensional embeddings from a pre-trained model like BERT or a vision transformer).

x = SpatialDropout1D(0.4)(inputs)

- **Purpose:** Applies dropout regularization right after the input to prevent overfitting by randomly "dropping" entire feature maps across the sequence dimension.

- **Mechanics:** SpatialDropout1D is a variant of dropout tailored for 1D sequences (like time series or text). Instead of dropping individual elements independently, it drops entire 1D channels (features) across all time steps in a correlated way. The rate 0.4 means 40% of the feature maps are set to zero during training (but scaled up by $1/(1-0.4) = 1.666$ to maintain expected activation magnitude). During inference, no dropout occurs.

x = Dense(512, activation='elu')(x) # ← NEW: 3072 → 512

- **Purpose:** A fully connected (dense) layer that projects the high-dimensional input features (3072) down to a lower dimension (512), acting as a bottleneck for dimensionality reduction and feature compression.
- **Mechanics:** For each time step in the sequence, it applies a linear transformation followed by the ELU (Exponential Linear Unit) activation: $\text{elu}(x) = x$ if $x \geq 0$ else $\alpha * (\exp(x) - 1)$, where $\alpha=1$ by default. This is done independently for each of the 24 time steps, so the layer treats the input as $(\text{batch_size} * 24, 3072)$ internally and reshapes back.
 - Trainable parameters: $(3072 \text{ input features} + 1 \text{ bias}) * 512 \text{ outputs} = 3072 * 512 + 512 = 1,573,376 + 512 = 1,573,888$.
- **Activation 'elu':** ELU is similar to ReLU but allows negative values, helping with the vanishing gradient problem in deep nets and promoting mean activations near zero for faster learning.
- **Comment '# ← NEW: 3072 → 512':** Indicates this is a recent addition, likely to reduce computational load from the original high dimensionality.
- **Implications:** Dimensionality reduction early on saves memory and compute in subsequent layers. Output shape: $(\text{batch_size}, 24, 512)$.

x = Conv1D(filters=512, kernel_size=3, padding='same', activation='elu', kernel_regularizer=regularizers.L2(l2=1e-3))(x)

- **Purpose:** Applies a 1D convolution to extract local patterns across the sequence (e.g., short-term dependencies or n-grams in text-like data).
- **Mechanics:** Slides 512 filters (kernels) of size 3 over the sequence dimension. Each filter computes a dot product with 3 consecutive time steps.
 - padding='same': Adds padding so output length remains 24 (input length).
 - Activation: ELU, as above.
 - kernel_regularizer=regularizers.L2(l2=1e-3): Adds L2 penalty (weight decay) to the kernel weights during optimization, penalizing large weights by adding $0.001 * \sum(\text{weights}^2)$ to the loss. This encourages smaller, more generalizable weights.
 - Trainable parameters: $(\text{kernel_size} * \text{input_channels} + 1 \text{ bias}) * \text{filters} = (3 * 512 + 1) * 512 = (1536 + 1) * 512 = 1537 * 512 \approx 786,944$.

- **Why Conv1D?:** Efficient for capturing local features before feeding into recurrent layers, acting as a "feature extractor" to make LSTMs focus on higher-level abstractions.
- **Implications:** Output shape: (batch_size, 24, 512). The L2 reg helps with overfitting in high-capacity models.

x = Dropout(0.4)(x)

- **Purpose:** Standard dropout after the LSTMs to further regularize by randomly zeroing 40% of elements.
- **Mechanics:** Independent dropout on each element (unlike SpatialDropout1D). Output shape unchanged: (batch_size, 24, 256).
- **Implications:** Helps prevent co-adaptation of features from the recurrent layers.

x = SeqSelfAttention()(x)

- **Purpose:** Adds a self-attention mechanism to weigh important parts of the sequence dynamically.
- **Mechanics:** This is likely from the keras-self-attention library (not standard Keras). It computes attention scores between all pairs of time steps, allowing the model to focus on relevant positions (e.g., key events in a time series). It outputs a weighted sum or attended sequence, typically preserving shape: (batch_size, 24, 256).
- **Implications:** Enhances LSTMs by adding global context (LSTMs are local/sequential). Trainable params depend on implementation (usually a few dense layers for queries/keys/values).

x = GlobalMaxPooling1D()(x)

- **Purpose:** Aggregates the sequence by taking the maximum value across the 24 time steps for each feature.
- **Mechanics:** Reduces the sequence dimension to 1, outputting (batch_size, 256). Max pooling captures the most salient features (e.g., peak activations).
- **Implications:** Converts variable-length sequence info (though fixed here) into a fixed vector for dense layers. No params.

The **inverted-pyramid (funnel) shape** used in the recurrent part of the model — **Bidirectional LSTM(1024) → 512 → 256 → 128** (i.e., **largest → smallest**) — is actually the **opposite** of the classic “pyramid” people usually talk about in CNNs. In sequence classification and sentiment analysis, this **inverse pyramid** (wide → narrow) is **extremely common and highly effective** for very specific reasons.

Here are the **real, practical benefits** you get from this design (especially for sentiment analysis, text classification, video/action classification, etc.):

Benefit	Why it matters in classification / sentiment analysis
Maximum context capture at the beginning	The first BiLSTM with 1024 units (→ 2048-dim hidden states) can store an enormous amount of bidirectional context from the entire sequence (24 steps in your case). This is crucial because sentiment or intent often depends on subtle interactions between distant words/frames. A huge hidden state acts like a "super memory" that remembers everything.
Progressive information compression & abstraction	Later layers (512 → 256 → 128) gradually distill the most relevant signals and discard noise. This mimics how humans go from "reading every word" → "forming an overall impression" → "making a final judgment". This hierarchical compression is exactly what you want before the final classifier.
Reduces risk of losing important long-range signals early	If you started narrow (e.g., 128 → 256 → 512 → 1024), early layers would have very limited capacity and might forget crucial context before later layers can recover it. Starting wide ensures nothing important is lost too soon.
Works perfectly with attention & pooling	You have SeqSelfAttention() right after the last (narrow) LSTM, followed by GlobalMaxPooling1D. A wide early representation gives attention much richer keys/queries/values to work with → attention can pick the truly important moments much better than if everything was already squeezed into 128 dimensions early.
Prevents gradient vanishing in very deep recurrent nets	Wider layers = more parameters per time step = stronger, richer gradients flowing back through time. This helps training stability in deep stacked BiLSTMs (you have 4 layers + attention).
Empirically proven in top-performing models	This exact inverted pyramid is used in:

Full modern Roberta Model for Sentimental Classification

```
def tokenize_function(examples):  
    tokenized = tokenizer(  
        examples["text"],  
        truncation=True,  
        padding="max_length",  
        max_length=128,  
    )  
    tokenized["labels"] = examples["category"]  
    return tokenized
```

- examples is a batch (dict of lists) from the dataset.
- truncation=True: cuts texts longer than 128 tokens
- padding="max_length": pads all sequences to exactly 128 tokens

```
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, predictions)
    f1 = f1_score(labels, predictions, average='weighted')
    precision = precision_score(labels, predictions, average='weighted')
    recall = recall_score(labels, predictions, average='weighted')
    return {
        "accuracy": acc,
        "f1_weighted": f1,
        "precision_weighted": precision,
        "recall_weighted": recall,
    }
```

- We take `argmax` for predicted class
- Uses **weighted average** : solves class imbalance issue

Example: if 90% of data is class 0, macro-F1 would punish poor performance on rare classes, weighted rewards overall correctness.

9. Training Arguments (Hyperparameters)

```
Python
training_args = TrainingArguments(
    output_dir="twitter_sentiment_final",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=16,
    num_train_epochs=6,
    weight_decay=0.001,
    load_best_model_at_end=True,
    metric_for_best_model="f1_weighted",
    greater_is_better=True,
    logging_steps=50,
    fp16=True,
    report_to="none",
    seed=42,
)

# where checkpoints/logs saved
# evaluate at end of each epoch
# save checkpoint each epoch
# standard for fine-tuning Transformers
# small due to GPU memory (RoBERTa-base)
# eval can be larger

# L2 regularization
# ← crucial with early stopping
# which metric to monitor
# obvious
# log every 50 steps
# mixed precision → 2x faster, less mem
# disable WandB/MLflow etc.
# reproducibility
```

10. Trainer Setup with Early Stopping

```
Python
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset,
    tokenizer=tokenizer,                    # for proper padding during inference la
    compute_metrics=compute_metrics,
    callbacks=[
        EarlyStoppingCallback(
            early_stopping_patience=1,      # stop if no improvement after 1 eval
            early_stopping_threshold=0.001  # minimum improvement to count as "better
        )
    ],
)
```

Early Stopping Details:

- Monitors `f1_weighted`
- If it doesn't improve by at least 0.001 for 1 consecutive evaluation (i.e., 1 epoch), training stops
- Combined with `load_best_model_at_end=True` → final model is the **best** one seen, not the last